Rapport du Projet de Machine Learning Retail

Frikha Amin GI2 S4

Table des matières

1 Introduction **2**

2 Contexte du projet **2**

3 Données et Exploration **2**
3.1 Source des données 2
3.2 Description des variables 2
3.3 Analyse Exploratoire (EDA) 3

4 Prétraitement des données **3**

5 Modélisation et Algorithmes **4**
5.1 Régression Logistique 4
5.2 XGBoost (Extreme Gradient Boosting) 4

6 Entraînement **4**

7 Évaluation et Performance **4**

8 Déploiement et Application Web **5**
8.1 Interface Utilisateur 5
8.2 Architecture Technique 5

9 Résultats et Perspectives **5**

10 Conclusion **6**

1 Introduction

Dans un marché de plus en plus concurrentiel, la rétention des clients est devenue une priorité stratégique pour les entreprises du secteur du "Retail". Le phénomène de "churn" (attrition), qui correspond à la perte de clients au profit de la concurrence, représente un manque à gagner significatif. Il est établi qu'acquérir un nouveau client coûte nettement plus cher que d'en fidéliser un existant.

Ce rapport présente une approche basée sur le Machine Learning pour anticiper le départ des clients. L'objectif est de construire un système capable de prédire, à partir des comportements passés, si un client est susceptible de rompre sa relation avec l'enseigne. Nous détaillons ici l'intégralité du cycle de vie du projet : de l'exploration des données transactionnelles au déploiement d'une solution opérationnelle via une interface web.

2 Contexte du projet

Le projet s'inscrit dans une démarche de "Data-Driven Marketing". En exploitant les traces numériques laissées par les consommateurs (achats, fréquence de visite, montants dépensés), nous cherchons à identifier des motifs comportementaux précurseurs d'un désengagement.

Le jeu de données utilisé regroupe des informations transactionnelles agrégées par client. L'enjeu technique réside dans la capacité du modèle à distinguer les fluctuations normales de consommation des signaux réels de churn. Une détection précoce permet à l'équipe marketing de déclencher des campagnes de réactivation ciblées (promotions personnalisées, sondages de satisfaction), optimisant ainsi le budget marketing.

3 Données et Exploration

3.1 Source des données

Les données brutes proviennent du système CRM de l'entreprise. Elles sont stockées dans le répertoire `data/` sous format CSV. Le fichier `data/transactions.csv` constitue notre source primaire après une étape initiale de nettoyage.

3.2 Description des variables

Le tableau ci-dessous détaille les caractéristiques (*features*) sélectionnées pour la modélisation :

Variable	Description détaillée
`client_id`	Identifiant unique (non utilisé pour la prédiction)
`total_achats`	Volume cumulé des transactions sur les 12 derniers mois
`valeur_moyenne`	Panier moyen du client en euros
`frequence_achat`	Nombre moyen de transactions par mois
`dernier_achat`	Récence : nombre de jours depuis la dernière activité
`churn`	**Cible** : 1 si le client a quitté l'enseigne, 0 sinon

Table 1 – Dictionnaire des données

3.3 Analyse Exploratoire (EDA)

L'analyse exploratoire a révélé une forte corrélation entre la récence (`dernier_achat`) et la probabilité de churn. Nous avons également observé que les clients ayant une fréquence d'achat élevée sont généralement plus fidèles, bien que leur panier moyen puisse être inférieur à la moyenne globale. Une attention particulière a été portée au déséquilibre des classes, le churn ne concernant qu'une minorité de clients.

4 Prétraitement des données

La qualité du modèle dépend directement de la préparation des données. Le script `src/utils.py` centralise les transformations suivantes :

- **Nettoyage** : Suppression des doublons et gestion des valeurs aberrantes (outliers) sur le panier moyen via la méthode des quartiles.
- **Imputation** : Les valeurs manquantes, bien que rares, ont été complétées par la médiane pour préserver la distribution.
- **Feature Engineering** : Création de ratios (ex : montant total / fréquence) pour capturer l'intensité de la relation client.
- **Scaling** : Utilisation du `StandardScaler` pour normaliser les variables numériques, garantissant que les modèles (comme la régression logistique) ne soient pas biaisés par les échelles de grandeur.
- **Splitting** : Division des données en 80% pour l'entraînement et 20% pour le test, avec une stratification sur la variable cible pour maintenir la proportion de churn dans les deux jeux.

Bien que la Figure **??** (pipeline visuel) ait été retirée du document final, les notebooks dans `notebooks/` détaillent chaque étape de transformation.

5 Modélisation et Algorithmes

Nous avons adopté une approche comparative pour sélectionner le modèle le plus performant.

5.1 Régression Logistique

Utilisée comme *baseline*, elle permet d'établir un seuil de performance minimal et d'analyser l'influence linéaire des variables. Sa simplicité garantit une grande interprétabilité, mais elle peine à capturer les relations complexes et non-linéaires.

5.2 XGBoost (Extreme Gradient Boosting)

XGBoost a été choisi pour sa puissance de calcul et sa capacité à traiter les relations non-linéaires. Il s'appuie sur un ensemble d'arbres de décision construits de manière séquentielle, où chaque nouvel arbre corrige les erreurs des précédents.

L'optimisation des hyperparamètres (nombre d'estimateurs, profondeur maximale des arbres, taux d'apprentissage) a été réalisée par `GridSearchCV`. Cette étape a permis de maximiser le score F1 tout en évitant le surapprentissage (*overfitting*).

6 Entraînement

Le processus d'entraînement est implémenté dans le script `src/train.py`. Le code utilise la bibliothèque `scikit-learn` et enregistre le modèle sérialisé dans le répertoire `models/` sous le nom `churn`$_{m}odel.pkl$.

7 Évaluation et Performance

`Les performances ont été évaluées sur le jeu de test indépendant. Nous avons privilégié le score F1 et le Rappel (`*`Recall`*`) car, dans un contexte de churn, il est plus coûteux de rater un client qui part (faux négatif) que de cibler par erreur un client fidèle (faux positif).`

Métrique	Score obtenu (XGBoost)
Exactitude (Accuracy)	0.88
Précision	0.82
Rappel (Recall)	0.84
F1-Score	**0.83**
AUC-ROC	0.91

Table 2 – Performances du modèle final

L'AUC-ROC de 0.91 indique une excellente capacité du modèle à séparer les deux classes. La matrice de confusion montre que le modèle identifie correctement 84% des cas de churn réels.

8 Déploiement et Application Web

Pour rendre le modèle utilisable par les équipes métier, nous avons développé une application web légère avec Flask.

8.1 Interface Utilisateur

L'interface permet de saisir manuellement les données d'un client. Un tableau de bord dynamique affiche ensuite la prédiction (Fidèle vs Risque de Churn) accompagnée d'un score de probabilité. Cette transparence permet aux agents de juger du niveau d'urgence de l'intervention.

8.2 Architecture Technique

L'application charge le modèle sérialisé (.pkl) lors du démarrage. Elle expose une route /predict qui accepte des requêtes POST contenant les données client au format JSON. Cette structure permet une intégration facile avec d'autres systèmes d'information (CRM, applications mobiles).

9 Résultats et Perspectives

Les résultats obtenus confirment que le comportement d'achat (fréquence et récence) est le meilleur prédicteur de la fidélité. Le modèle XGBoost offre une robustesse satisfaisante pour une mise en production initiale.

Cependant, plusieurs pistes d'amélioration sont envisageables :

- Données temporelles : Intégrer des séries temporelles pour capturer l'évolution du comportement au fil des mois plutôt que des moyennes fixes.
- Segmentation : Appliquer un clustering (ex: K-Means) avant la prédiction pour affiner les modèles par segment de clientèle (VIP, occasionnels, etc.).
- Explicabilité : Utiliser des outils comme SHAP ou LIME pour expliquer chaque prédiction individuelle aux équipes marketing.

10 Conclusion

Ce projet a permis de mettre en place une chaîne de traitement complète, illustrant la puissance du Machine Learning pour résoudre des problématiques business concrètes. En automatisant la détection du churn, l'entreprise peut passer d'une stratégie de rétention réactive à une stratégie proactive, préservant ainsi sa base client et son chiffre d'affaires. L'intégration de cette solution dans le workflow quotidien des équipes marketing constitue la prochaine étape logique pour transformer ces analyses en valeur économique réelle.

Références

[1] Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR 12, pp. 2825-2830, 2011.

[2] Chen, T., Guestrin, C., *XGBoost: A Scalable Tree Boosting System*, KDD 2016.

[3] Grinberg, M., *Flask Web Development*, O'Reilly Media, 2018.